A Call to Action: Transitioning Signal's Private Group System to Quantum-Safe

Graeme Connell[1], *Sebastian Faller*[2,3], Felix Günther[2], Julia Hesse[2], Vadim Lyubashevsky[2], *Rolfe Schmidt*[1]

1 Signal

2 IBM Research Europe

3 ETH zürich

Group Messaging

There is a lot of interesting work on encrypting group messages.

Users also need to *manage* the group.

If management is not secure, the chat is not secure.

Group Management

Product designers create **access rules** such as:

- “Only current group members can learn group membership”
- “Only administrators can remove someone from a group”
- “You can only add someone to a group if you are already connected to them on the service”

Then we build a system to enforce those rules.

Group Management

A standard way to accomplish much of this would be

- Group data in a database
- A server enforces the rules
- All protected with standard perimeter security

This requires full trust in service administrators.

And enormous trust in cloud providers!

This is not acceptable for Signal.

We must minimize server trust and enforce as much as possible cryptographically.

There is more: Signal wants to protect group membership data. We don’t want to know who is connected to whom.

Signal Private Group System

Signal Private Groups:

Keep the DB, but encrypt it.

To Make This Work

We need

- **Anonymous Credentials** so users cannot be linked between lists
- **Verifiable Encryption** so the Group Server can ensure that it contains valid data
- **Zero Knowledge Proofs** to link the two together

[CCS:CPZ20] - https://eprint.iacr.org/2019/1416

So, about quantum safety...

The Quantum Safety Problems

Quantum Safe Long Term

We need a migration plan.

The

- Anonymous Credentials
- Verifiable Encryption
- ZK proofs

in today's system are all Elliptic Curve based.

They are not quantum safe.

HNDL Now

We want to take action soon.

It is not just authentication, there are Harvest Now Decrypt Later (HNDL) vulnerabilities.

- The encrypted group DB is not HNDL secure
- Credential presentations and ZK proofs are not HNDL secure

Signal’s wishlist for a private group system

We must make changes, let’s try to get everything we want:

- Quantum safe replacements for our primitives
- With reasonable costs
- Straightforward hybridization
- And a nice cheap HNDL option today

Then we can get started on a smooth, incremental migration

You can’t always get what you want

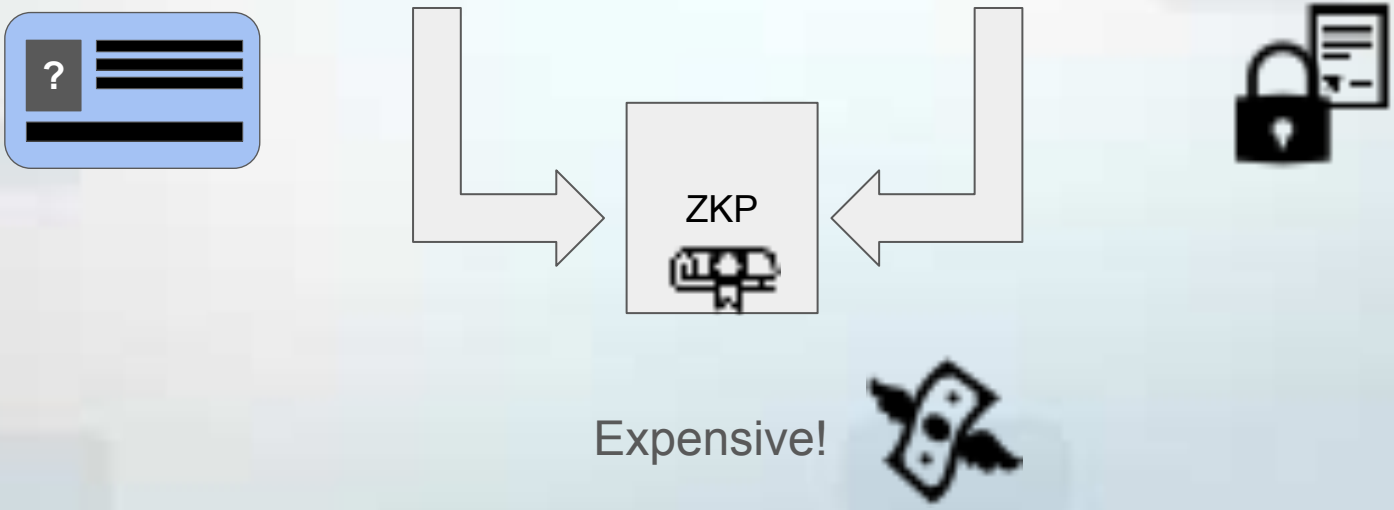
Lattice-based Verifiable Encryption, ZKPs, and KVACs

Lattice-based anonymous credentials

~30KB Size, ~110KB comm. [BLNS23]

Verifiable encryption, e.g., with [LNP22]

~ 20KB proof size



Hybrid Security out of reach for current system

- Hybrid verifiable encryption:

  `Lattice-Enc(DH-Enc(m))` or `DH-Enc(Lattice-Enc(m))`

- Either requires a ZK-proof over both encryption schemes!

Instantiating current system quantum-safely?

- Quantum safe replacements for our all primitives ✗
- With reasonable costs ✗
- Straightforward hybridization ✗
- Cheap HNDL option today ?

Does the server have to validate everything?

Does the server have to validate everything?

- The server verifying properties on encrypted data is expensive
- Can we let the user verify some of the properties?
- Maybe this allows us to get away without the verifiable encryption

Client Validation?

Ok, maybe. But…

The server should still be able to perform important validation

- Clients should prove to the server that they are authorized to read or update the group
  - Even if they can’t prove that the update is fully valid
- Clients should be able to prove to the server that group members are registered Signal users.

Client Validation

If clients have to validate ciphertexts they need to be able to

- Identify faulty or misbehaving clients
- Identify and reject or ignore bad ciphertexts
  - Can this be abused?
- Clean up garbage

So can we get this sort of control if we accept client-side validation?

If clients can validate, we have new possibilities

Powerful shift: Users check validity

Once the users check validity, we can simplify the system a lot!

Powerful shift: Users check validity

Once the users check validity, we can simplify the system a lot!

Let's start with a over-simplified protocol.

Signatures instead of credentials

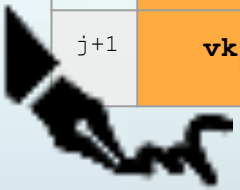
Server's Group List		
Nr	SigKey	Ciphertext
1	vk_1	$Enc(gk, (UID_1, prk_1))$
		…
i	vk_i	$Enc(gk, (UID_i, prk_i))$
		…
j	vk_j	$Enc(gk, (UID_j, prk_j))$
j+1	vk_{j+1}	$Enc(gk, UID_{j+1}, prk_{j+1})$

Signatures instead of credentials

- Signature key pair per user
- User signs group list
- Server and users can verify

Server's Group List

Nr	SigKey	Ciphertext
1	vk_1	$Enc(gk, (UID_1, prk_1))$
		…
i	vk_i	$Enc(gk, (UID_i, prk_i))$
		…
j	vk_j	$Enc(gk, (UID_j, prk_j))$
j+1	vk_{j+1}	$Enc(gk, UID_{j+1}, prk_{j+1})$



Signed by i-th user

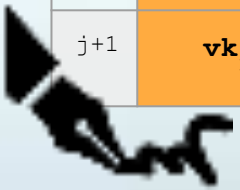
Signatures instead of credentials

- Signature key pair per user
- User signs group list
- Server and users can verify

Problem: Linkability.

Server's Group List

Nr	SigKey	Ciphertext
1	vk_1	$Enc(gk, (UID_1, prk_1))$
		…
i	vk_i	$Enc(gk, (UID_i, prk_i))$
		…
j	vk_j	$Enc(gk, (UID_j, prk_j))$
j+1	vk_{j+1}	$Enc(gk, UID_{j+1}, prk_{j+1})$



Signed by i-th user

Signatures instead of credentials

- Signature key pair per user
- User signs group list
- Server and users can verify

Problem: Linkability.

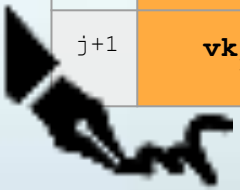
Need per group vk → **pseudonym**.

Blaming: All members must derive $\mathbf{vk}_i$.

Privacy: But not the server.

Server's Group List

Nr	SigKey	Ciphertext
1	$\mathbf{vk}_1$	$\mathrm{Enc}(gk, (UID_1, prk_1))$
		…
i	$\mathbf{vk}_i$	$\mathrm{Enc}(gk, (UID_i, prk_i))$
		…
j	$\mathbf{vk}_j$	$\mathrm{Enc}(gk, (UID_j, prk_j))$
j+1	$\mathbf{vk}_{j+1}$	$\mathrm{Enc}(gk, UID_{j+1}, prk_{j+1})$



Signed by i-th user

Signatures instead of credentials

- Signature key pair per user
- User signs group list
- Server and users can verify

Problem: Linkability.

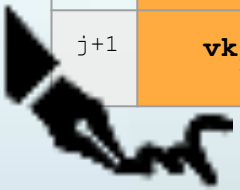
Need per group vk → **pseudonym**.

Blaming: All members must derive $\mathbf{vk}_i$.

Privacy: But not the server.

Server's Group List

Nr	SigKey	Ciphertext
1	$\mathbf{vk}_1$	$\mathrm{Enc}(gk, (UID_1, prk_1))$
		…
i	$\mathbf{vk}_i$	$\mathrm{Enc}(gk, (UID_i, prk_i))$
		…
j	$\mathbf{vk}_j$	$\mathrm{Enc}(gk, (UID_j, prk_j))$
j+1	$\mathbf{vk}_{j+1}$	$\mathrm{Enc}(gk, UID_{j+1}, prk_{j+1})$



Signed by i-th user

Solution: Key-Rerandomizable Signatures [FKM+16]

Key-Rerandomizable Signatures? (classically) [FKM+16]

- Derive a pseudonym for each group

```
base-sk = x
```

$$\texttt{base-vk} = g^x$$

Key-Rerandomizable Signatures? (classically) [FKM+16]

- Derive a pseudonym for each group

$$\texttt{base-sk} = x$$

$$\texttt{base-vk} = g^x$$

Long term, publicly known

Key-Rerandomizable Signatures? (classically) [FKM+16]

- Derive a pseudonym for each group

per group seed

```
r = H(UID,rrs)
```

```
base-sk = x
```

$$\texttt{base-vk} = g^x$$

Long term, publicly known

Key-Rerandomizable Signatures? (classically) [FKM+16]

- Derive a pseudonym for each group

only known to user

per group seed

$$\texttt{r = H(UID,rrs)}$$

$$\texttt{base-sk = x} \qquad \texttt{rand-sk = x+r}$$

$$\texttt{base-vk = } g^x \qquad \texttt{pseudonym = } g^x * g^r$$

Long term, publicly known

Known to all group members

Key-Rerandomizable Signatures? (classically) [FKM+16]

- Derive a pseudonym for each group
- Group members can derive the randomizer **`r`** from UID and rrs
- Group members learn the pseudonym
- Only the owner of **`base-sk`** knows **`rand-sk`**

only known to user

per group seed

`r = H(UID,rrs)`

`base-sk = x`

`base-vk =` g^x

`rand-sk = x+r`

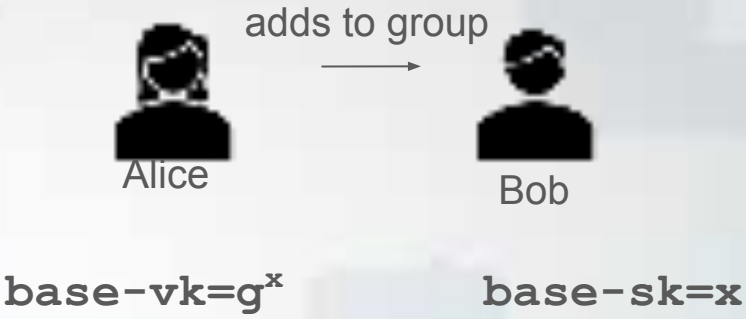
`pseudonym =` $g^x * g^r$

Long term, publicly known

Known to all group members

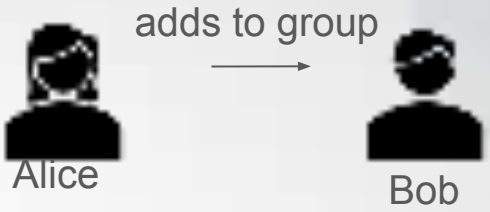
Adding someone to the group

- Get the **`base-vk`** of the new user
- Rerandomize **`base-vk`** with **`r=H(UID,rrs)`**
- Store pseudonym in the group list
- The new user can now sign under **`rand-sk`**



Adding someone to the group

- Get the **`base-vk`** of the new user
- Rerandomize **`base-vk`** with **`r=H(UID,rrs)`**
- Store pseudonym in the group list
- The new user can now sign under **`rand-sk`**

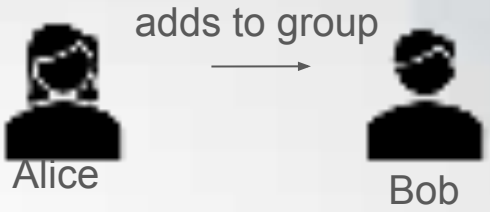


$\texttt{base-vk}=g^x$ $\texttt{base-sk}=x$

$\texttt{r} = \texttt{H}(\texttt{UID}_{\texttt{Bob}},\texttt{rrs})$

Adding someone to the group

- Get the **`base-vk`** of the new user
- Rerandomize **`base-vk`** with **`r=H(UID,rrs)`**
- Store pseudonym in the group list
- The new user can now sign under **`rand-sk`**



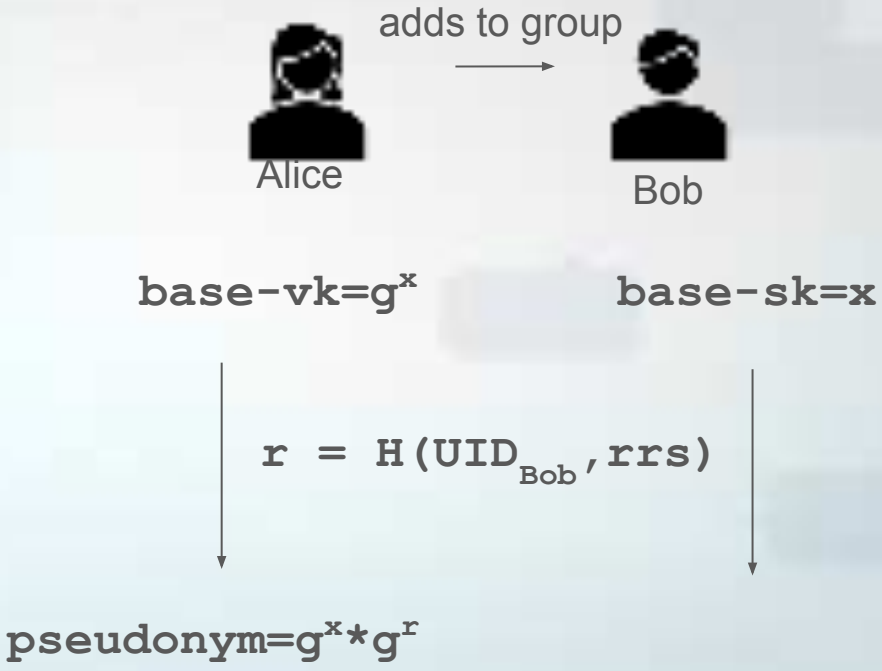
$\texttt{base-vk}=g^x$ $\texttt{base-sk}=x$

$\texttt{r} = \texttt{H}(\texttt{UID}_{\texttt{Bob}},\texttt{rrs})$

$\texttt{pseudonym}=g^x*g^r$

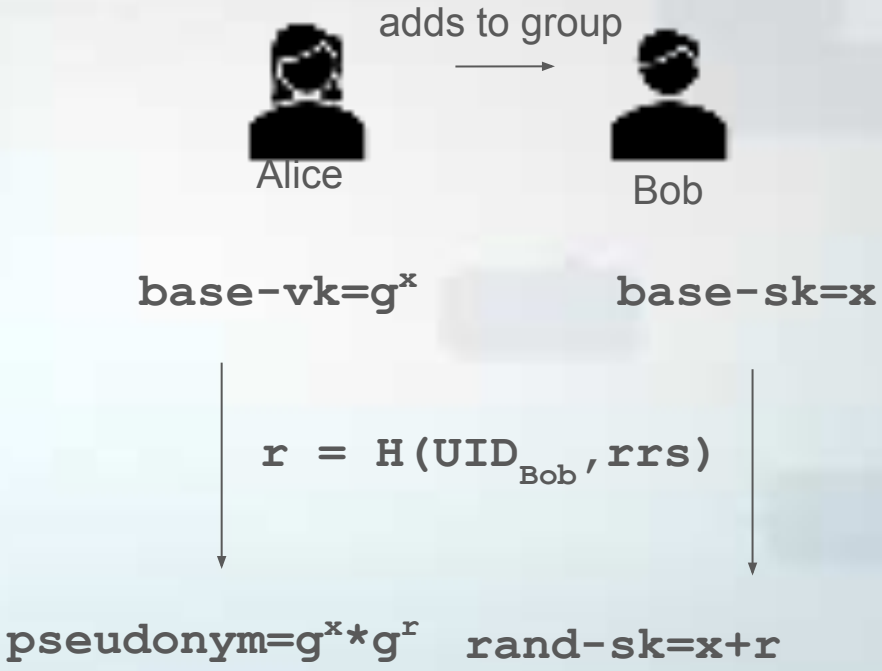
Adding someone to the group

- Get the **`base-vk`** of the new user
- Rerandomize **`base-vk`** with **`r=H(UID,rrs)`**
- Store pseudonym in the group list
- The new user can now sign under **`rand-sk`**



Adding someone to the group

- Get the **`base-vk`** of the new user
- Rerandomize **`base-vk`** with **`r=H(UID,rrs)`**
- Store pseudonym in the group list
- The new user can now sign under **`rand-sk`**



Key-Rerandomizable Signatures from Dilithium

```
base-sk = x

base-vk = g^x

r = H(UID,rrs)


rand-sk = x+r

pseudonym = g^x * g^r
```

→

```
base-sk = x (small norm)

base-vk = Ax

r = H(UID,rrs)

s.t. r small norm


rand-sk = x+r

pseudonym = Ax+Ar
```

Key-Rerandomizable Signatures from Dilithium

```
base-sk = x

base-vk = g^x

r = H(UID,rrs)


rand-sk = x+r

pseudonym = g^x * g^r
```

→

```
base-sk = x (small norm)

base-vk = Ax

r = H(UID,rrs)

s.t. r small norm


rand-sk = x+r

pseudonym = Ax+Ar
```

- Larger `rand-sk = x+r` (2x)
- We still want same sign. size
- More rejections (x4). Expected 20 samples needed
- Same security as Dilithium. Same verification algorithm

More functionality

- Key Rotation
- Invitations
- Distinguish user and admin privileges
- We can still use the commitments for lost devices. But much rarer.

Did we get what we wanted?

- Cheap HNDL option today ✓
  - Symm. enc. + Dlog-based key-rerand. sig.
- Clear path for quantum-safe replacements ✓
  - Dilithium-based key-rerand. sigs.
- With reasonable costs ✓
  - ~3KB for Dilithium-based, 64B for Dlog-based
- Straightforward hybridization ✓
  - Use both signatures

Did we get what we wanted?

Cheap HNDL option today

- Symm. enc. + Dlog-based key-rerand. sig.

Clear path for quantum-safe replacements

- Dilithium-based key-rerand. sigs.

With reasonable costs

- ~3KB for Dilithium-based, 64B for Dlog-based

Straightforward hybridization

- Use both signatures

Switch was possible due to ground-up protocol redesign

Problem solved.
So why a “call to action”?

Group Management is a Small Preview

With group messaging we are migrating a **system**

- Not a protocol
- Not a bag of primitives

We have

- Complex app-specific auth and confidentiality rules
- Serious scalability and performance requirements

Details will be different, but many of you are facing similar challenges.

We’re learning from experience

This project took a different direction from what I expected.

But by

- Focusing on **system** goals
- Designing with **available** tools

We have

- Solved immediate (HNDL) problems
- Paved a clear path to quantum safety
- Maybe even made things more efficient

We don’t think our experience will be unique.

Thank you!

Signal

IBM

ETH zürich

This is an HNDL Problem

Verifiable Encryption

VerEnc(UID; a_1,a_2):

M_1 = HashToPoint(UID)
M_2 = EncToPoint(UID)
$E_1 = [a_1]M_1$
$E_2 = [a_2]E_1 + M_2$

return (E_1,E_2)

A attacker who

- Gets a copy of the database
- Can guess a ctext/ptext pair in a group
- Gets a quantum computer…

…Can compute the group secret and read everything

Credential Presentations

Presentations include:

$C_{x0} = [z]\, G_{x0} + U$
$C_{x1} = [z]\, G_{x1} + [t]\, U$
$C_{y0} = [z]\, G_{y0}$ // for expiration date
$C_{y1} = [z]\, G_{y1} +$ HashToPoint(UID)
$C_{y2} = [z]\, G_{y2} +$ EncodeToPoint(UID)
$C_V = [z]\, G_V + V$

An attacker who

- Records a credential presentation
- Gets a quantum computer…

…can recover the hidden attributes and the credential.

This is an HNDL Problem

Verifiable Encryption

VerEnc(UID; a_1,a_2):

M_1 = HashToPoint(UID)
M_2 = EncToPoint(UID)
$E_1 = [a_1]M_1$
$E_2 = [a_2]E_1 + M_2$

return (E_1,E_2)

A attacker who

- Gets a copy of the database
- Can guess a ctext/ptext pair in a group
- Gets a quantum computer…

…Can compute the group secret and read everything

Get **z** here and then peel off everything else.

Credential Presentations

Presentations include:

$C_{x0} = [z]\, G_{x0} + U$
$C_{x1} = [z]\, G_{x1} + [t]\, U$
$C_{y0} = [z]\, G_{y0}$ // for expiration date
$C_{y1} = [z]\, G_{y1} + \text{HashToPoint}(UID)$
$C_{y2} = [z]\, G_{y2} + \text{EncodeToPoint}(UID)$
$C_V = [z]\, G_V + V$

An attacker who

- Records a credential presentation
- Gets a quantum computer…

…can recover the hidden attributes and the credential.